

AI4Everyone

Corso di Formazione su Claude

Manuale Utente — Terza Lezione

Data sessione: 30 marzo 2026

Docente: Stefano Zaghi

Partecipanti: Partecipante 1, Partecipante 2, Partecipante 3, Partecipante 4

Piattaforma: Microsoft Teams | Durata: ~2 ore

Sommario

1. INTRODUZIONE.....	4
1.1 Panoramica della Lezione	4
1.2 Obiettivi di Apprendimento.....	4
2. RIEPILOGO DELLA SECONDA LEZIONE	5
2.1 Architettura RAG e Gestione della Memoria	5
Come funziona l'architettura RAG	5
Proprietà fondamentale della memoria in Claude.....	5
3. IL METAPROMPTING	6
3.1 Definizione e Principio	6
3.2 Come si Applica.....	6
3.3 Esempio Pratico: "Context is All You Need"	6
4. CLAUDE CODE: INTRODUZIONE E INSTALLAZIONE.....	7
4.1 Cos'è Claude Code.....	7
L'Orchestratore e il suo System Prompt	7
4.2 Installazione e Requisiti	7
4.3 Login e Autenticazione	8
4.4 Interfaccia e Primo Avvio	8
5. COMANDI PRINCIPALI	9
5.1 Tabella di Riferimento.....	9
5.2 /context — Monitoraggio del Contesto	10
5.3 /compact — Gestione del Contesto	11
Soglie di compattazione automatica.....	11
5.4 /resume — Riprendere una Conversazione Precedente	12
5.5 /config — Pannello di Configurazione	12
5.6 /model — Selezione del Modello	13
6. MODALITÀ OPERATIVE	14
6.1 Modalità Standard (Ask Permission).....	14
6.2 Modalità Auto-Accept Edits.....	14
6.3 Plan Mode (Modalità Pianificazione).....	15
Piano automatico vs. Piano manuale	15
7. CLAUDE.MD — CONFIGURAZIONE DEL PROGETTO.....	16
7.1 Cos'è il File CLAUDE.md.....	16
7.2 Analogia con Claude Desktop.....	16
7.3 Come Generare il File CLAUDE.md	17
7.4 Dove Vengono Salvati i Dati di Claude Code.....	17
8. MEMORY.MD — MEMORIA DINAMICA.....	18
8.1 Cos'è il Memory.md	18
8.2 Come Funziona	18

9. FLUSSO DI LAVORO: DA CLAUDE DESKTOP A CLAUDE CODE	19
9.1 Ambienti Separati	19
9.2 Il CLAUDE.md come collegamento.....	19
9.3 Quando Usare l'Uno e Quando l'Altro	20
10. PIANO DI SVILUPPO E TASK ATOMICI	21
10.1 Perché Pianificare Prima di Agire	21
10.2 Struttura di un Piano di Sviluppo.....	21
10.3 Come Eseguire i Task	22
10.4 Revisione e Iterazione	22
11. ESERCIZIO PRATICO: "CONTEXT IS ALL YOU NEED"	23
11.1 Obiettivo dell'Esercizio.....	23
11.2 Sequenza di Passaggi Eseguiti	23
12. INTRODUZIONE ALLO SPEC-DRIVEN DEVELOPMENT	24
12.1 Il Cambiamento di Paradigma.....	24
12.2 Il Ruolo Centrale delle Specifiche	24
12.3 Anticipazione della Prossima Lezione.....	24
13. RIEPILOGO	25
13.1 Concetti Chiave della Lezione	25
13.2 Esercizio Assegnato	25
13.3 Glossario	26

1. INTRODUZIONE

1.1 Panoramica della Lezione

Questa terza sessione del percorso formativo su Claude Desktop e Claude Code rappresenta un importante momento di transizione: dopo aver esplorato la gestione dei progetti Claude e l'architettura RAG nelle lezioni precedenti, si inizia oggi a costruire ponti concreti tra la fase di pianificazione — tipicamente svolta in Claude Desktop — e la fase di implementazione, affidata a Claude Code.

L'obiettivo centrale della lezione è comprendere come trasformare un'idea in un prototipo di soluzione adottando un approccio metodico e strutturato. A tal fine vengono introdotti due strumenti distinti ma complementari: il file `CLAUDE.md` come dispositivo di configurazione dell'agente in Claude Code, e il concetto di piano di sviluppo come primo tassello della metodologia Spec-Driven Development.

Per rendere i concetti immediatamente tangibili, tutta la parte pratica è sviluppata attorno a un esercizio volutamente giocoso: la composizione di una canzone intitolata "Context is All You Need", ispirata ai Beatles e rivisitata in chiave AI. La scelta di un dominio non tecnico (la musica) è deliberata: dimostra che Claude Code — spesso associato esclusivamente allo sviluppo software — è in realtà uno strumento di ragionamento e produzione di output applicabile a qualsiasi ambito.

1.2 Obiettivi di Apprendimento

Al termine di questa lezione il partecipante sarà in grado di:

- Comprendere la differenza strutturale tra Claude Desktop e Claude Code e scegliere lo strumento più adatto in base alla fase di lavoro.
- Installare Claude Code, effettuare il login con SSO aziendale e navigare l'interfaccia da terminale.
- Utilizzare i comandi principali di Claude Code: `/exit`, `/resume`, `/context`, `/compact`, `/clear`, `/config`, `/model`.
- Comprendere il funzionamento della context window, il meccanismo di auto-compattazione e come gestirlo consapevolmente.
- Creare e strutturare il file `CLAUDE.md` come configurazione di base di un progetto.
- Distinguere tra `CLAUDE.md` (wiki di progetto) e `memory.md` (taccuino dinamico dell'agente).
- Generare un piano di sviluppo strutturato e scomporlo in task atomici sequenziali.
- Utilizzare le tre modalità operative di Claude Code: `standard`, `auto-accept edits`, `plan mode`.

2. RIEPILOGO DELLA SECONDA LEZIONE

2.1 Architettura RAG e Gestione della Memoria

La seconda lezione ha introdotto la gestione avanzata dei progetti in Claude Desktop con un focus particolare sull'architettura RAG (Retrieval-Augmented Generation) e sui meccanismi di memoria persistente. Questi concetti costituiscono la base teorica indispensabile per comprendere le scelte progettuali di Claude Code.

Come funziona l'architettura RAG

Quando si caricano documenti nella sezione "Files" di un progetto Claude Desktop, questi non vengono ingeriti in blocco all'interno del contesto della conversazione. Il processo che si attiva è più sofisticato:

1. I documenti vengono suddivisi in frammenti più piccoli denominati chunk.
2. Ciascun chunk viene vettorializzato: trasformato in un vettore numerico che ne codifica il significato semantico.
3. I vettori vengono indicizzati in un database vettoriale, una struttura dati specializzata nel calcolo della similarità semantica.
4. Quando si invia un prompt, il sistema calcola la distanza vettoriale tra la richiesta e i chunk indicizzati, recuperando i frammenti concettualmente più vicini.
5. Solo i chunk pertinenti vengono inseriti nel contesto della conversazione per formulare la risposta.

Nota

Un'analogia intuitiva: se "latte + uova = torta", un database vettoriale sa che queste parole sono concettualmente correlate anche in assenza di un legame grammaticale diretto. La distanza vettoriale misura la vicinanza concettuale, non quella lessicale.

Proprietà fondamentale della memoria in Claude

Un principio cardine che permea tutto il corso è che ogni nuova conversazione parte da un contesto vuoto. Gli agenti Claude non hanno memoria intrinseca: ogni sessione ricomincia da zero. Costruire un progetto con istruzioni e documenti serve esattamente a questo: fornire all'agente una base documentale strutturata dalla quale ripartire ad ogni conversazione, senza dover ricominciare ogni volta da zero.

3. IL METAPROMPTING

3.1 Definizione e Principio

Il metaprompting è una tecnica che consiste nel chiedere a Claude di generare il miglior prompt per ottenere un determinato output, invece di tentare di scrivere direttamente quel prompt da soli. Si tratta di delegare all'AI la progettazione del prompt ottimale, sfruttandone la conoscenza del proprio funzionamento.

Il principio è semplice: l'AI conosce meglio di noi come interpretare una richiesta in modo efficace. Fornendole il contesto dell'obiettivo che vogliamo raggiungere, possiamo ottenere un prompt strutturato, completo e ottimizzato che — eseguito successivamente — produce output di qualità nettamente superiore rispetto a un prompt scritto "a mano" senza metodo.

3.2 Come si Applica

La sequenza operativa del metaprompting è la seguente:

1. Definire chiaramente l'obiettivo: cosa si vuole ottenere come output finale.
2. Scrivere un prompt di meta-richiesta: "Sono X. Il mio obiettivo è Y. Genera il miglior prompt in italiano per ottenere i seguenti contenuti: [lista output desiderati]".
3. Lasciare che Claude costruisca il prompt ottimale — spesso utilizzando la skill Prompt Architect dell'organizzazione.
4. Esaminare il prompt generato, eventualmente modificarlo, poi eseguirlo in una nuova chat o nel progetto dedicato.

3.3 Esempio Pratico: "Context is All You Need"

Durante l'esercizio della lezione il docente ha applicato il metaprompting per generare tre artifact di configurazione del progetto in un'unica conversazione: descrizione sintetica, descrizione dettagliata e system prompt (istruzioni di progetto).

Il meta-prompt inviato era strutturato come segue:

```
Sei un esperto di musica ma anche di Generative e Agentic AI.  
L'obiettivo del progetto è comporre un brano musicale intitolato  
"Context is All You Need", rivisitazione di "All You Need Is Love"  
dei Beatles, in chiave moderna (pop, soft rock, rap).
```

```
Genera il miglior prompt in italiano per produrre:
```

- ```
1. Una descrizione sintetica del progetto
2. Una descrizione dettagliata ed esaustiva
3. Le istruzioni di progetto (system prompt)
```

#### Suggerimento

Perché generare i tre artifact in un'unica conversazione e non in tre separate? Lavorando in un unico contesto, Claude sa cosa ha scritto in ciascun documento e non si contraddice. Con conversazioni separate, potrebbe inserire informazioni incoerenti tra loro.

## 4. CLAUDE CODE: INTRODUZIONE E INSTALLAZIONE

### 4.1 Cos'è Claude Code

Claude Code è un'implementazione di Claude sviluppata da Anthropic per ambienti di sviluppo e terminale. A differenza di Claude Desktop — che opera attraverso un'interfaccia web o mobile — Claude Code viene eseguito direttamente da riga di comando e porta con sé un workflow agentico preconfigurato, ottimizzato per attività che richiedono ragionamento strutturato, produzione di file e gestione del filesystem locale.

È importante comprendere che, benché sia nato come strumento per lo sviluppo software, Claude Code si presta a qualsiasi compito che richieda: produzione di output strutturati, gestione di file, esecuzione di piani a più passi, coordinamento di sotto-agenti. La scelta dell'esempio musicale durante la lezione è intenzionale: dimostrare che Claude Code non è un tool per soli sviluppatori.

### L'Orchestratore e il suo System Prompt

Quando si avvia Claude Code, l'interlocutore non è un'AI generica: è un agente specifico denominato Main Orchestrator Agent. Questo agente ha un proprio system prompt scritto da Anthropic, ottimizzato per coordinare task complessi e, se necessario, delegarli a sotto-agenti specializzati. Diversamente dal system prompt di un progetto Claude Desktop, questo non può essere modificato dall'utente.

### 4.2 Installazione e Requisiti

Claude Code viene distribuito come pacchetto npm e richiede Node.js installato sulla macchina. Il comando di installazione globale è il seguente.

**Windows Powershell:**

```
irm https://claude.ai/install.ps1 | iex
```

**macOS, Linux, WSL:**

```
curl -fsSL https://claude.ai/install.sh | bash
```

#### Nota

È necessario che Node.js sia già installato. Verificare con: `node --version`  
Per il piano aziendale Team AI4Everyone non sono necessari ulteriori abbonamenti separati.

Riferimenti: <https://code.claude.com/docs/en/quickstart>

## 4.3 Login e Autenticazione

Al primo avvio, Claude Code richiede l'autenticazione. Il processo è il seguente:

1. Avviare Claude Code dal terminale con il comando: `claude`
2. Selezionare la modalità di login: scegliere l'opzione 1 (piano con subscription aziendale).
3. Il sistema apre automaticamente il browser per il completamento del flusso OAuth.
4. Grazie al Single Sign-On (SSO) aziendale abilitato, non viene richiesta username né password: l'autenticazione è automatica.
5. Dare l'autorizzazione e tornare al terminale.
6. Al riavvio successivo, il login sarà già memorizzato.

### Suggerimento

Il comando per forzare un nuovo login manuale è: `/login`

Se si dispone di più account (es. piano Team aziendale e piano Max personale), al momento del login il browser proporrà quale account utilizzare.

## 4.4 Interfaccia e Primo Avvio

Una volta avviato, Claude Code mostra un'interfaccia conversazionale da terminale del tutto simile a quella di Claude Desktop, ma con caratteristiche aggiuntive:

- In basso a destra viene visualizzato il numero di token consumati nella conversazione corrente.
- Sono disponibili comandi speciali preceduti dal carattere `/` (slash).
- Il sistema mostra le operazioni che sta eseguendo (lettura file, scrittura, chiamate ad agenti).
- Per le modifiche ai file viene chiesta esplicitamente l'approvazione dell'utente (modalità standard).

### Attenzione

Claude Code deve essere avviato all'interno della cartella su cui si vuole lavorare.

La cartella di avvio diventa automaticamente il "progetto" di quella sessione.

Conversazioni e file generati verranno mantenuti associati a quella cartella.



## 5. COMANDI PRINCIPALI

Claude Code mette a disposizione un insieme di comandi invocabili con il prefisso / (slash). Di seguito una descrizione dettagliata dei comandi più importanti per il lavoro quotidiano.

### 5.1 Tabella di Riferimento

| Comando                            | Descrizione                                                                                         |
|------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>/exit</code>                 | Esce da Claude Code e torna alla console del sistema operativo                                      |
| <code>/login</code>                | Avvia il flusso di autenticazione (utile se non si è ancora loggati)                                |
| <code>/resume</code>               | Mostra le conversazioni precedenti della cartella corrente e permette di riprenderne una            |
| <code>/context</code>              | Visualizza lo stato corrente dell'utilizzo del contesto (token consumati, tool disponibili, agenti) |
| <code>/compact [istruzioni]</code> | Compatta la cronologia della conversazione in un riassunto, liberando spazio nel contesto           |
| <code>/clear</code>                | Pulisce completamente la cronologia della conversazione senza riassumere                            |
| <code>/config</code>               | Apri il pannello di configurazione interattivo di Claude Code                                       |
| <code>/model</code>                | Permette di cambiare il modello AI da usare nella sessione corrente                                 |
| <code>Shift + Tab</code>           | Scorciatoia da tastiera per ciclare tra le modalità operative (standard, auto-accept, plan mode)    |

## 5.2 /context — Monitoraggio del Contesto

---

Il comando `/context` è uno dei più utili per chi vuole mantenere il controllo sul proprio flusso di lavoro. Mostra lo spazio occupato, nella context window, dai seguenti elementi:

- System prompt dell'orchestratore (fornito da Anthropic).
- Tool disponibili, agenti configurati, skill installate.
- File di memoria (es. `CLAUDE.md`, `memory.md`) già caricati nel contesto.
- Percentuale di riempimento della context window.

In basso a destra dell'interfaccia viene mostrato il numero di token consumati. Con il modello Claude Opus la context window è di 1.000.000 token; con Claude Sonnet, in alcune configurazioni, può essere più limitata (es. 200K token). Tenere sempre d'occhio questo indicatore è buona pratica, soprattutto nelle sessioni di lavoro lunghe.

### Nota

Claude Code inizia ogni sessione con un contesto di base occupato dal proprio system prompt, dagli strumenti disponibili e — se presente — dal file `CLAUDE.md` della cartella corrente. Questo "costo iniziale" è sempre presente e va tenuto in considerazione.

## 5.3 /compact — Gestione del Contesto

---

Quando la conversazione diventa molto lunga e il contesto si sta riempiendo, è possibile utilizzare il comando /compact per compattare la storia della conversazione in un riassunto. Questo libera spazio nel contesto permettendo di continuare a lavorare.

Il comando accetta un parametro opzionale con istruzioni per la compattazione:

```
Compattazione base
/compact

Compattazione guidata con priorità specifiche
/compact Metti in risalto le decisioni architetturali prese
```

### Soglie di compattazione automatica

Claude Code gestisce in autonomia la compattazione quando il contesto supera una certa soglia. Il sistema funziona così:

- Al 50% di occupazione del contesto, il sistema inizia a mostrare avvisi visivi e la percentuale rimanente invece del numero assoluto di token.
- Al 65% di occupazione, Claude Code si riserva il 35% restante come "auto-compact buffer": uno spazio dedicato esclusivamente per eseguire la compattazione automatica.
- Se il buffer viene raggiunto, la compattazione parte automaticamente senza intervento dell'utente.

#### **Attenzione**

Il riassunto prodotto dalla compattazione è frequentemente impreciso e può causare la perdita di dettagli significativi. La best practice raccomandata è: salvare il lavoro fatto, aprire una nuova conversazione (/clear o /exit + rientro) e riprendere da lì.

## 5.4 /resume — Riprendere una Conversazione Precedente

Ogni conversazione avviata in una cartella viene salvata automaticamente da Claude Code nella directory di sistema dell'utente (Windows:

C:\Users\[username]\AppData\Roaming\Claude\projects\). Il comando /resume permette di selezionare e ripristinare una sessione precedente esattamente dal punto in cui era stata interrotta, con tutto il contesto conservato.

```
Aprire Claude Code nella cartella del progetto
cd C:\percorso\al\progetto
claude

Per riprendere una sessione precedente
/resume

Selezionare la sessione desiderata dalla lista
```

### Nota

Importante: /resume mostra solo le sessioni associate alla cartella corrente, non tutte le conversazioni avute con Claude Code su qualsiasi cartella. Ogni cartella è un contenitore di sessioni indipendente.

## 5.5 /config — Pannello di Configurazione

Il comando /config apre un pannello interattivo con le opzioni di configurazione di Claude Code. Tra le impostazioni più rilevanti:

- **Thinking mode:** se abilitato, il modello "pensa di più" prima di rispondere. Aumenta il consumo di token ma migliora significativamente la qualità degli output. Raccomandato: sempre abilitato.
- **Auto-compact:** possibilità di disabilitare la compattazione automatica (utile in casi specifici di debug).
- **Modello di default:** impostare il modello che Claude Code usa automaticamente all'avvio.

### Best Practice

Tenere sempre abilitato il Thinking Mode. Il costo in token aggiuntivi è ampiamente compensato dal miglioramento qualitativo degli output, specialmente per task complessi.

## 5.6 /model — Selezione del Modello

---

Claude Code permette di cambiare il modello AI durante una sessione. È possibile passare da Claude Opus (più potente, context window più ampia) a Claude Sonnet (più veloce, più economico) e viceversa.

### **Attenzione**

Ogni modello mantiene il proprio contesto separato: passare da Opus a Sonnet e poi ritornare a Opus NON trasferisce la conversazione. I due contesti sono indipendenti e non comunicano. È come parlare con due agenti distinti nella stessa sessione.

Per questa ragione, la best practice suggerita dal docente è di non cambiare modello durante una sessione di lavoro. La gestione di più modelli con ruoli diversi (es. Opus per la pianificazione, Sonnet per l'implementazione) va demandata all'architettura multi-agente di Claude Code, che verrà approfondita in sessioni future.

## 6. MODALITÀ OPERATIVE

Claude Code offre tre modalità operative distinte che determinano il livello di autonomia concesso all'agente nel modificare file e produrre output. La modalità attiva è sempre visibile in basso a sinistra nell'interfaccia e può essere cambiata rapidamente con la scorciatoia da tastiera Shift + Tab.

### 6.1 Modalità Standard (Ask Permission)

È la modalità predefinita. In questa configurazione Claude Code chiede esplicitamente l'approvazione dell'utente prima di ogni operazione rilevante: creazione di file, modifica di file esistenti, esecuzione di comandi.

Per ogni modifica proposta, il sistema mostra un diff (le differenze) tra lo stato attuale e quello che vorrebbe produrre. L'utente può rispondere:

- Sì / Y: approva la singola modifica e va avanti.
- No: rifiuta la modifica e può fornire una motivazione o richiedere un approccio diverso.
- "Allow all" / "Always allow": approva tutte le modifiche rimanenti senza ulteriori conferme (da usare con cautela).

#### Best Practice

La modalità standard è quella raccomandata per chi sta imparando o lavora su task critici. Permette di mantenere il pieno controllo sull'output e correggere il tiro in tempo reale, senza aspettare che tutto sia completato per accorgersi di un errore.

### 6.2 Modalità Auto-Accept Edits

In questa modalità, Claude Code applica automaticamente tutte le modifiche senza chiedere conferma. È utile quando si ha piena fiducia nel piano di lavoro definito e si vuole velocizzare l'esecuzione di task che richiedono molte operazioni di scrittura.

#### Attenzione

Usare con cautela. In modalità auto-accept, errori o interpretazioni errate del prompt vengono applicati automaticamente senza possibilità di intervenire. Conviene usarla solo dopo aver verificato il piano di sviluppo e su task ben circoscritti.

## 6.3 Plan Mode (Modalità Pianificazione)

Il Plan Mode è una modalità speciale dedicata alla generazione dei piani di sviluppo. Quando attivato, Claude Code si comporta in modo notevolmente diverso:

1. Esplora la cartella/codebase attuale per acquisire contesto sul progetto.
2. Analizza il prompt ricevuto e verifica se mancano informazioni per procedere.
3. Se necessario, presenta un menù interattivo con domande guidate (opzioni selezionabili) per raccogliere le informazioni mancanti.
4. Genera un piano di sviluppo strutturato come file temporaneo (non salvato immediatamente nella cartella di progetto).
5. Al termine della pianificazione, chiede se procedere immediatamente all'esecuzione. Possiamo eventualmente richiedere il salvataggio del piano di sviluppo al fine di lavorarci, ossia di iniziare l'attività, in una nuova conversazione.

Per attivare il Plan Mode si può usare la scorciatoia Shift + Tab oppure inserire nel prompt la seguente stringa trigger:

```
Trigger testuale per Plan Mode
Usa la modalità pianificazione (Use planning mode)
```

### Suggerimento

Best practice per la pianificazione:

1. Usare Plan Mode con il modello Claude Opus per ottenere piani di sviluppo di qualità superiore.
  2. Salvare il piano come file nella cartella di progetto prima di avviare l'esecuzione.
  3. Iniziare l'esecuzione in una nuova conversazione, caricando il piano come riferimento.
- Questo mantiene il contesto pulito e l'output più focalizzato.

### Piano automatico vs. Piano manuale

Durante la lezione è stato mostrato un confronto tra un piano generato con un prompt strutturato (senza Plan Mode) e uno generato con Plan Mode. La qualità del secondo è risultata decisamente superiore, grazie al processo di raccolta informazioni guidata e alla logica di ragionamento più profonda attivata dalla modalità dedicata.

Il piano prodotto in Plan Mode include: obiettivo del progetto, struttura dell'output atteso, vincoli stilistici e tecnici, scelte prese e relative motivazioni, e infine i task di sviluppo con i dettagli necessari per lavorarci in sessioni separate.

## 7. CLAUDE.MD — CONFIGURAZIONE DEL PROGETTO

### 7.1 Cos'è il File CLAUDE.md

Il file CLAUDE.md è il meccanismo principale di configurazione dei progetti in Claude Code. Si tratta di un file di testo in formato Markdown (estensione .md) che deve essere posizionato nella cartella radice del progetto. Quando Claude Code viene avviato in una cartella contenente un CLAUDE.md, il file viene caricato automaticamente nel contesto di base dell'agente prima ancora che l'utente scriva il primo prompt.

Il CLAUDE.md svolge la stessa funzione delle "Instructions" nei progetti di Claude Desktop: definisce l'identità dell'agente, l'obiettivo del progetto, i vincoli da rispettare, le convenzioni da seguire, e qualsiasi altro contesto stabile che deve essere sempre presente nell'agente durante tutta la sessione.

### 7.2 Analogia con Claude Desktop

| Elemento (Claude Desktop)    | Equivalente (Claude Code)                              |
|------------------------------|--------------------------------------------------------|
| Instructions / System Prompt | CLAUDE.md                                              |
| Files / Project Knowledge    | File nella cartella di progetto (letti esplicitamente) |
| Progetto                     | Cartella di lavoro                                     |

#### Nota

Differenza cruciale: a differenza di Claude Desktop, in Claude Code gli altri file presenti nella cartella NON vengono caricati automaticamente nel contesto. Solo CLAUDE.md è automatico. Per tutti gli altri file, bisogna istruire esplicitamente l'agente a leggerli nel prompt.



## 7.3 Come Generare il File CLAUDE.md

Il metodo ottimale per creare un CLAUDE.md ben strutturato è sfruttare il metaprompting all'interno di un progetto Claude Desktop già configurato:

1. Aprire il progetto Claude Desktop che già contiene le istruzioni e la knowledge base del progetto.
2. Inviare un prompt che chiede di generare il miglior CLAUDE.md per quel progetto, specificando lo scopo (uso in Claude Code) e la struttura attesa.
3. Salvare l'artifact generato come file CLAUDE.md nella cartella di lavoro locale.
4. Avviare Claude Code puntando a quella cartella: il file verrà caricato automaticamente.

### Suggerimento

Vantaggi di questo approccio:

- Si parte da un progetto già ricco di istruzioni e contesto.
- Il CLAUDE.md generato è coerente con la knowledge base del progetto Desktop.
- L'agente in Code partirà con un contesto di alta qualità già strutturato.

## 7.4 Dove Vengono Salvati i Dati di Claude Code

Claude Code salva automaticamente tutte le sue impostazioni, sessioni e output in una cartella dedicata all'utente del sistema operativo:

```
Windows
C:\Users\[username]\.Claude\

Struttura interna
Claude\
├── projects\ ← sessioni per cartella di progetto
│ ├── d-test\ ← conversazioni del progetto "d:\test"
│ ├── d-task-management\
│ └── ...
├── tasks\ ← task svolti dai sotto-agenti
└── plans\ ← piani di sviluppo generati in Plan Mode
```

I file di conversazione sono in formato JSON e contengono l'intera storia dei messaggi scambiati. Non è necessario consultarli direttamente: è possibile chiedere a Claude Code di recuperare informazioni dallo storico attraverso il normale prompt conversazionale.

## 8. MEMORY.MD — MEMORIA DINAMICA

### 8.1 Cos'è il Memory.md

Oltre al CLAUDE.md, Claude Code gestisce un secondo tipo di file di memoria: il memory.md. Se il CLAUDE.md è il "wiki stabile" del progetto — un documento di configurazione che cambia raramente — il memory.md è il "taccuino dinamico" dell'agente: un file che viene aggiornato in tempo reale per registrare informazioni specifiche che l'agente deve ricordare per le sessioni future.

### 8.2 Come Funziona

Il memory.md viene creato e aggiornato dall'agente quando l'utente gli chiede esplicitamente di memorizzare qualcosa, oppure quando l'agente lo ritiene necessario. Ad esempio:

```
Esempi di istruzioni che scatenano la scrittura su memory.md
Ricordati che gestisco git autonomamente, non eseguire mai comandi git
da solo.
Da ora in poi, apri sempre le pagine in un nuovo tab, non nella stessa
finestra.
```

Il file viene solitamente creato, nell'area dedicata alle impostazioni di progetto (C:\Users\[username]\Claude\projects\...), in una sottocartella memory/.

Il file memory.md contiene frammenti di informazione in formato testo libero o strutturato, organizzati per task o per preferenza utente.

#### Suggerimento

Sia CLAUDE.md che memory.md sono file di testo leggibili e modificabili.

Se si vuole cancellare un'informazione memorizzata, si può semplicemente eliminare la riga corrispondente nel file memory.md — o chiedere a Claude Code di farlo.

## 9. FLUSSO DI LAVORO: DA CLAUDE DESKTOP A CLAUDE CODE

### 9.1 Ambienti Separati

Claude Desktop e Claude Code sono due ambienti distinti e non comunicano direttamente. Non esiste un meccanismo automatico per trasferire le istruzioni di un progetto Desktop in una sessione di Claude Code. Ogni ambiente ha la propria struttura di configurazione e il proprio spazio di memoria.

Questo non significa che non si possano usare insieme: il `CLAUDE.md` è esattamente il ponte progettuale che permette di trasportare il contesto da un ambiente all'altro. È un'operazione manuale, ma semplice e ben definita.

### 9.2 Il `CLAUDE.md` come collegamento

Il flusso completo per passare da Claude Desktop a Claude Code è il seguente:

1. Configurare il progetto in Claude Desktop: istruzioni, knowledge base, contesto ricco.
2. Usare il metaprompting all'interno del progetto Desktop per chiedere la generazione del `CLAUDE.md`.
3. Scaricare il `CLAUDE.md` come artifact e copiarlo nella cartella locale di lavoro.
4. Avviare Claude Code puntando a quella cartella: il file viene caricato automaticamente.
5. Procedere con la pianificazione e lo sviluppo in Claude Code.

#### Nota

In alternativa, si può creare il `CLAUDE.md` direttamente in Claude Code chiedendogli: "Generami il file `CLAUDE.md` per questo tipo di progetto".

Tuttavia, partire da un progetto Desktop già configurato produce un file di qualità superiore, poiché Claude ha accesso a tutte le istruzioni e la knowledge base già definite.

### 9.3 Quando Usare l'Uno e Quando l'Altro

| Strumento      | Quando usarlo                                         |
|----------------|-------------------------------------------------------|
| Claude Desktop | Fase di ideazione e brainstorming iniziale            |
| Claude Desktop | Costruzione e affinamento del contesto di progetto    |
| Claude Desktop | Conversazioni esplorative senza output su file system |
| Claude Desktop | Uso da mobile o senza accesso al terminale            |
| Claude Code    | Implementazione e sviluppo (produzione di file)       |
| Claude Code    | Task che richiedono accesso al filesystem locale      |
| Claude Code    | Flussi con approvazione granulare delle modifiche     |
| Claude Code    | Orchestrazione di più agenti e task paralleli         |

Workflow strutturato:

- 1) Claude Desktop: brainstorming, configurazione del progetto e del contesto
- 2) Passaggio da Claude Desktop a Claude Code: generazione, in Claude Desktop, del file di configurazione Claude.md per Claude Code
- 3) Generazione di un piano di sviluppo: può essere svolto in Claude Desktop e successivamente revisionato in Claude Code, oppure può essere effettuato direttamente in Claude Code (in questo caso, sfruttando il Plan Mode)
- 4) Lavorazione dei task in Claude Code.

 **Best Practice**

Per lavori strutturati: usare Claude Desktop per la fase upstream (ideazione, architettura, definizione del contesto), poi passare a Claude Code per l'implementazione con controllo granulare. Il CLAUDE.md è il file di passaggio tra i due ambienti.

## 10. PIANO DI SVILUPPO E TASK ATOMICI

### 10.1 Perché Pianificare Prima di Agire

---

Una delle abitudini più importanti da sviluppare nell'uso di Claude Code — e più in generale in qualsiasi flusso di lavoro AI — è quella di pianificare i task prima di eseguirli.

Chiedere all'agente di produrre direttamente l'output finale in un'unica sessione porta tipicamente a due problemi: riempimento prematuro del contesto (se il task è lungo) e output di qualità inferiore (se il task è complesso).

La soluzione è scomporre il lavoro in più task atomici sequenziali, ciascuno eseguito in una conversazione separata, con il proprio contesto pulito. Questo approccio è il cuore della metodologia Spec-Driven Development, di cui la lezione introduce i fondamenti.

### 10.2 Struttura di un Piano di Sviluppo

---

Un piano di sviluppo ben strutturato include:

- Obiettivo generale: cosa deve essere prodotto al termine di tutti i task.
- Struttura dell'output atteso: come deve essere organizzato il risultato finale.
- Vincoli e linee guida: regole da rispettare durante lo sviluppo.
- Task sequenziali numerati: ognuno con il proprio obiettivo specifico, input richiesti, output atteso, e informazioni sufficienti per essere eseguito in una nuova conversazione.
- Domande aperte: eventuali chiarimenti necessari prima di iniziare.

## 10.3 Come Eseguire i Task

---

Per ciascun task del piano, il flusso operativo raccomandato è:

1. Aprire una nuova conversazione in Claude Code (nella stessa cartella del progetto).
2. Il CLAUDE.md verrà caricato automaticamente come contesto di base.
3. Fornire nel prompt il riferimento al piano di sviluppo: "Leggi il piano contenuto nel file development\_plan.md e svolgi il task N."
4. Claude Code leggerà il file, eseguirà il task e produrrà l'output nella cartella di progetto.
5. Revisionare l'output, fornire feedback, eventualmente chiedere correzioni.
6. Una volta soddisfatti, uscire dalla conversazione e aprirne una nuova per il task successivo.

```
Esempio di prompt per eseguire un task
Leggi il piano di sviluppo contenuto nel file development_plan.md
e svolgi il task 1.
Qualora tu abbia dubbi, fammi delle domande prima di procedere.
```

### Suggerimento

Perché è importante specificare "qualora tu abbia dubbi, fammi delle domande"?  
Permette a Claude Code di segnalare informazioni mancanti prima di iniziare il lavoro, evitando che faccia assunzioni errate che invaliderebbero l'output.

## 10.4 Revisione e Iterazione

---

Al termine di ciascun task, Claude Code propone automaticamente una fase di review: mostra il risultato prodotto e chiede se si vuole procedere con il task successivo o apportare modifiche. Questo momento di revisione è prezioso e non va saltato.

Se si rilevano problemi nell'output, è possibile richiedere revisioni all'interno della stessa conversazione prima di passare al task successivo. Le revisioni consumano contesto, ma garantiscono che il materiale su cui si costruirà il prossimo task sia di qualità.

## 11. ESERCIZIO PRATICO: "CONTEXT IS ALL YOU NEED"

### 11.1 Obiettivo dell'Esercizio

---

L'esercizio della lezione ha usato la composizione di una canzone come pretesto per applicare in modo concreto tutti i concetti teorici introdotti: metaprompting, creazione di un progetto Claude strutturato, generazione del file di configurazione CLAUDE.md per il passaggio a Claude Code, redazione di un piano di sviluppo strutturato in task atomici, utilizzo del Plan Mode.

Il tema — "Context is All You Need", rivisitazione moderna di "All You Need Is Love" dei Beatles con influenze pop, soft rock e rap — è stato scelto deliberatamente per dimostrare che Claude Code funziona su qualsiasi dominio, non solo su quello del software.

### 11.2 Sequenza di Passaggi Eseguiti

---

1. Metaprompting in Claude Desktop per generare il prompt ottimale di configurazione.
2. Esecuzione del prompt con produzione di tre artifact: descrizione sintetica, descrizione dettagliata, system prompt.
3. Creazione del progetto "Context is All You Need" in Claude Desktop.
4. Metaprompting per generare il CLAUDE.md del progetto.
5. Copia del CLAUDE.md nella cartella locale di lavoro e avvio di Claude Code.
6. Generazione del piano di sviluppo in Claude Desktop, oppure direttamente in Claude Code sfruttando il Plan Mode (2–3 task).
7. Esecuzione del task 1 in una nuova conversazione (prima bozza del testo).
8. Review dell'output e eventuale iterazione.
9. Esecuzione del task 2 (raffinamento e versione definitiva).
10. [Opzionale] Generazione audio con Suno AI a partire dal testo prodotto.

## 12. INTRODUZIONE ALLO SPEC-DRIVEN DEVELOPMENT

### 12.1 Il Cambiamento di Paradigma

Nella storia tradizionale dello sviluppo software, si scrivono le specifiche funzionali e tecniche, poi il software. Completato lo sviluppo, il software diventa la "fonte di verità" del comportamento del sistema: è il codice, con tutto il suo dettaglio, a definire cosa il sistema fa realmente.

Le specifiche diventano documentazione storica, spesso disallineata dal codice effettivo.

Con l'avvento dei large language models e di strumenti come Claude Code, questo paradigma si sta invertendo. Oggi è possibile — e sempre più conveniente — scrivere specifiche di qualità elevata e delegare all'AI la produzione del codice. In questo scenario, la specifica torna ad essere la fonte di verità: è da essa che dipende la qualità del software prodotto.

### 12.2 Il Ruolo Centrale delle Specifiche

Il messaggio finale condiviso al termine della lezione è emblematico: "Il nostro lavoro sta diventando scrivere specifiche di qualità. Scrivendo delle specifiche qualitativamente migliori si ottiene un output migliore. I piani di sviluppo assumono un ruolo più importante di quanto pensiamo."

Le implicazioni pratiche di questo cambio di paradigma sono:

- Salvare i propri prompt e le proprie specifiche come se fossero asset preziosi.
- Costruire flussi di lavoro documentati e ripetibili.
- Rivedere e migliorare continuamente le specifiche: la loro qualità è il principale driver di qualità dell'output.
- Concentrare l'attenzione sulla chiarezza e completezza del "cosa si vuole" più che sul "come ottenerlo".

#### Best Practice

Salva sempre i tuoi prompt e i tuoi piani di sviluppo. Non sono file temporanei: sono la tua specifica, la tua fonte di verità. Migliorarli iterazione dopo iterazione è il modo più efficace per migliorare la qualità degli output AI nel tempo.

### 12.3 Anticipazione della Prossima Lezione

La quarta lezione approfondirà in modo sistematico la metodologia Spec-Driven Development, includendo i nuovi flussi guidati introdotti recentemente da Claude Code per supportare questo approccio.



## 13. RIEPILOGO

### 13.1 Concetti Chiave della Lezione

---

I concetti fondamentali introdotti in questa terza lezione sono:

1. Claude Code è un agente specializzato eseguibile da terminale, con workflow agentico preconfigurato, applicabile a qualsiasi dominio — non solo al coding.
2. Il file CLAUDE.md rappresenta la configurazione di base del progetto in Claude Code (equivalente alle Instructions di Claude Desktop). Viene caricato automaticamente ad ogni avvio nella cartella corrente.
3. Il file memory.md è il taccuino dinamico dell'agente: registra preferenze e informazioni utili per le sessioni future, su richiesta esplicita dell'utente.
4. La context window in Claude Code è monitorabile in tempo reale. La compattazione automatica parte al 65% di occupazione. Meglio gestirla manualmente prima di raggiungere quella soglia.
5. Le tre modalità operative (standard, auto-accept, plan mode) determinano il livello di autonomia dell'agente. Il plan mode è dedicato alla pianificazione strutturata.
6. Claude Desktop e Claude Code sono ambienti separati. Il CLAUDE.md è il ponte: si genera il file in Desktop e si porta in Code per iniziare le attività di sviluppo.
7. I task atomici e i piani di sviluppo sono il modo corretto di affrontare compiti complessi: pianificare prima, eseguire dopo, in sessioni separate.
8. Lo Spec-Driven Development porta a una inversione del paradigma: le specifiche diventano la fonte di verità. La qualità del lavoro si misura dalla qualità della specifica.

### 13.2 Esercizio Assegnato

---

Per questa sessione non è stato assegnato un esercizio strutturato. Il docente invita i partecipanti a:

- Installare Claude Code e familiarizzarsi con l'interfaccia da terminale.
- Esplorare le tre modalità operative: standard, auto-accept, plan mode.
- Usare Claude Code per qualsiasi task personale o lavorativo — anche non tecnico — per prendere confidenza con lo strumento.
- Non è richiesto nessun output specifico: l'obiettivo è l'esplorazione libera.

### 13.3 Glossario

| Termine                          | Definizione                                                                                                       |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>CLAUDE.md</code>           | File di configurazione del progetto in Claude Code. Caricato automaticamente all'avvio.                           |
| <code>memory.md</code>           | File di memoria dinamica dell'agente. Aggiornato su richiesta per ricordare preferenze e note.                    |
| <code>Context Window</code>      | Lo "spazio di lavoro" dell'agente. Contiene tutta la conversazione corrente e il contesto di base.                |
| <code>Auto-compact Buffer</code> | Spazio riservato da Claude Code per eseguire la compattazione automatica ( $\approx 16,5\%$ del contesto totale). |
| <code>Plan Mode</code>           | Modalità operativa dedicata alla generazione di piani di sviluppo strutturati con raccolta guidata di requisiti.  |
| <code>Metaprompting</code>       | Tecnica che consiste nel chiedere a Claude di generare il miglior prompt per un determinato obiettivo.            |
| <code>Task Atomico</code>        | Un'unità di lavoro singola, circoscritta e completa in sé, che può essere eseguita in una conversazione dedicata. |
| <code>Spec-Driven Dev.</code>    | Metodologia in cui le specifiche sono la fonte di verità e guidano l'implementazione AI.                          |
| <code>Thinking Mode</code>       | Opzione di configurazione che fa ragionare di più il modello prima di rispondere (token extra, output migliore).  |
| <code>Main Orchestrator</code>   | L'agente principale di Claude Code, con system prompt Anthropic, in grado di coordinare sotto-agenti.             |
| <code>Chunk</code>               | Frammento di documento prodotto dalla fase di indicizzazione nell'architettura RAG.                               |
| <code>RAG</code>                 | Retrieval-Augmented Generation. Architettura che recupera i chunk semanticamente rilevanti per rispondere.        |